

Indigenous Environmental Data

Science

Teacher Guide



About Us

These slides are a collaboration between:

[Victoria Thomson](#), an Albert Einstein Distinguished Educator Fellow with USGS and high school science teacher, Christine Martin, PhD Candidate at Montana State University, and Rachel Torres, a postdoctoral scholar at Cal Poly Humboldt

The larger project and coding activities were created through a collaboration between Georgia Smies, an instructor at Salish Kootenai College, Sean Dorr, Jason McLachlan, and students at the University of Notre Dame. It is funded by the Alfred P. Sloan Foundation.

Our goal is to increase equitable pathways in earth and environmental data science.



Time and Structure of Indigenous Environmental Data Science



Piloted at Salish Kootenai College in Montana Winter Quarter 2024
Course: Intro to R Statistics in the Wildlife and Fisheries Dept.

Task: Complete Modules 1 - 3



Each Module requires ~3 weeks (6 hr class time) + outside class time

Frequency of Meetings:

Meet 1x per week for 2 hours

Total of 10 weeks



Class Format:

First hour of class: direct instruction/discussion



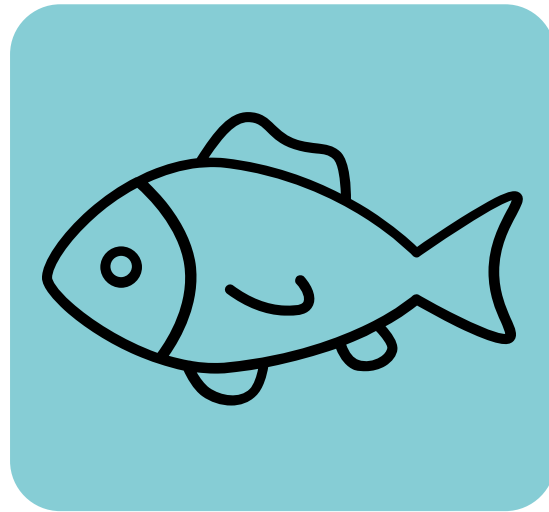
Background and Purpose



- Why are we doing this?
 - To introduce you to a fictional tribe K'avi so you can analyze data relevant to indigenous communities
 - Code can appear scary; but the sooner you play with it, you will see it's not so bad! It can be really powerful for you to know and understand how to use this programming language
 - To empower you to ask questions that are relevant to your community and identify how to share the information learned from your questions
- What is data science?
 - Data science is an interdisciplinary field that uses scientific methods, processes, algorithms, and systems to extract knowledge and insights from data (source: [datacamp](#))
- What is R?
 - R is a programming language commonly used to analyze large sets of data



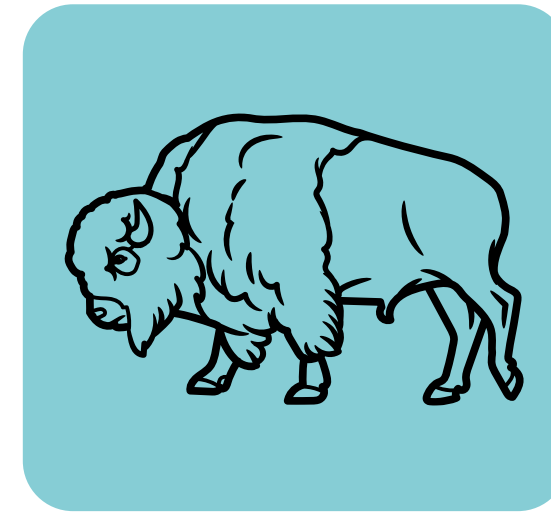
Indigenous Environmental Data Science Modules



Module 1: Mercury
in Fish



Module 2: Water
Quality



Module 3: Bison
Reintroduction



Module 4: Forestry
Management



[Github](#)



GitHub is like a big online notebook where people can save and share their coding projects.

Imagine you're working on a school project with friends. Instead of passing a notebook back and forth, GitHub lets you all work on it at the same time and saves every change.

It helps people:

- ✓ Store their code safely
- ✓ Work together on projects
- ✓ Keep track of all changes

Think of it as Google Docs, but for coding! 🚀



[Environmental Indigenous Data Science
Code on Github](#)

Choose Your Path



[R and R studio](#)
[Free, but requires](#)
[downloading software](#)

[Best for:](#)
[Laptops with memory](#)
[Computer labs](#)

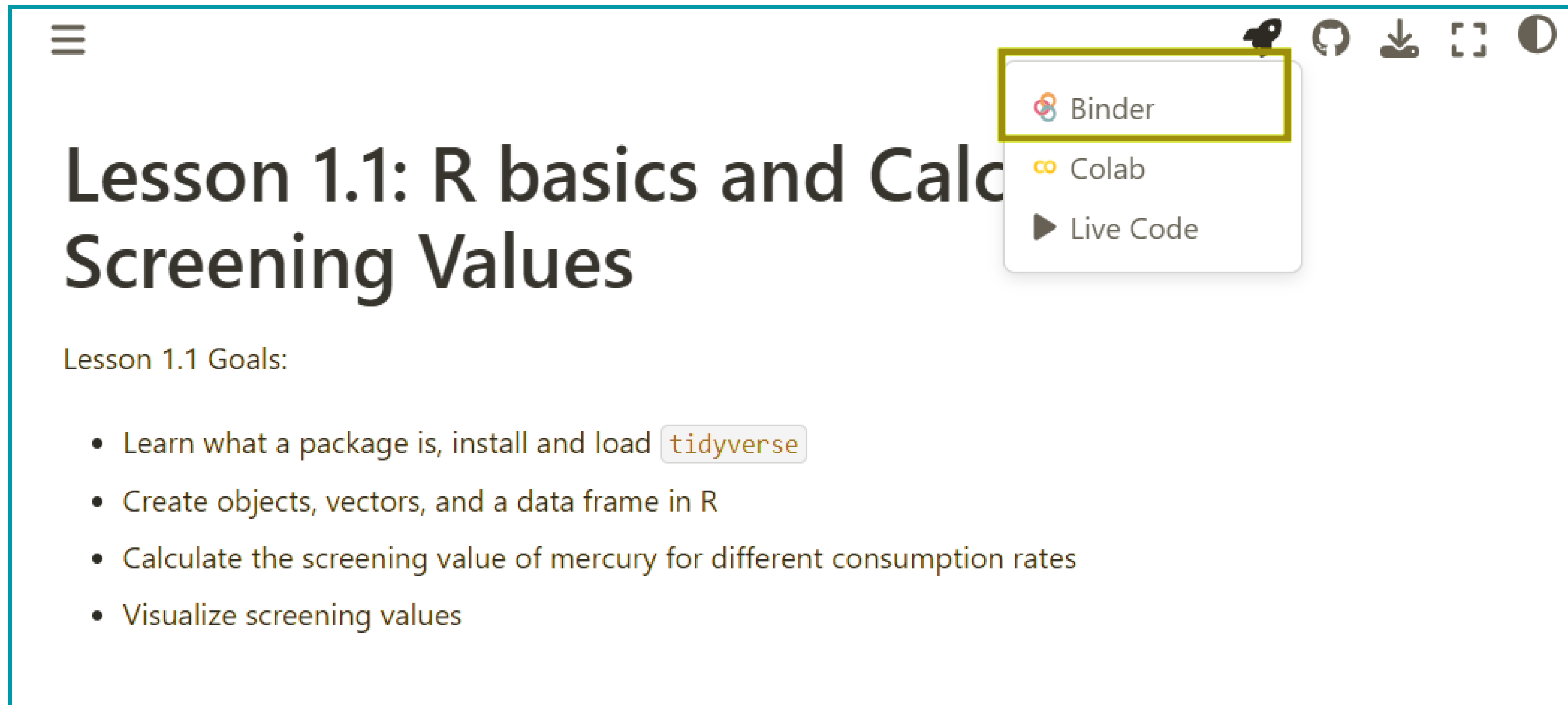


[binder](#)
[Free, cloud-based service](#)
[to run notebooks](#)

[Best for:](#)
[Any environment](#)

Using Binder to Run Jupyter with R: access by hovering over the rocket icon

Clicking Binder will open a new tab and start to load (this can take a few minutes)



The screenshot shows a web interface for a lesson titled "Lesson 1.1: R basics and Calculating Screening Values". The interface includes a header with a hamburger menu icon on the left and a toolbar on the right with icons for a rocket (Binder), a circular arrow (refresh), a download arrow, a full-screen icon, and a moon icon (dark mode). A dropdown menu is open over the rocket icon, showing three options: "Binder" (with the Binder logo), "Colab" (with the Colab logo), and "Live Code" (with a play button icon). The "Binder" option is highlighted with a yellow border. Below the title, the text "Lesson 1.1 Goals:" is followed by a bulleted list of four goals. The first goal mentions installing and loading the "tidyverse" package, which is shown in a light blue box. The other goals are about creating objects, calculating screening values for mercury, and visualizing the results.

Lesson 1.1: R basics and Calculating Screening Values

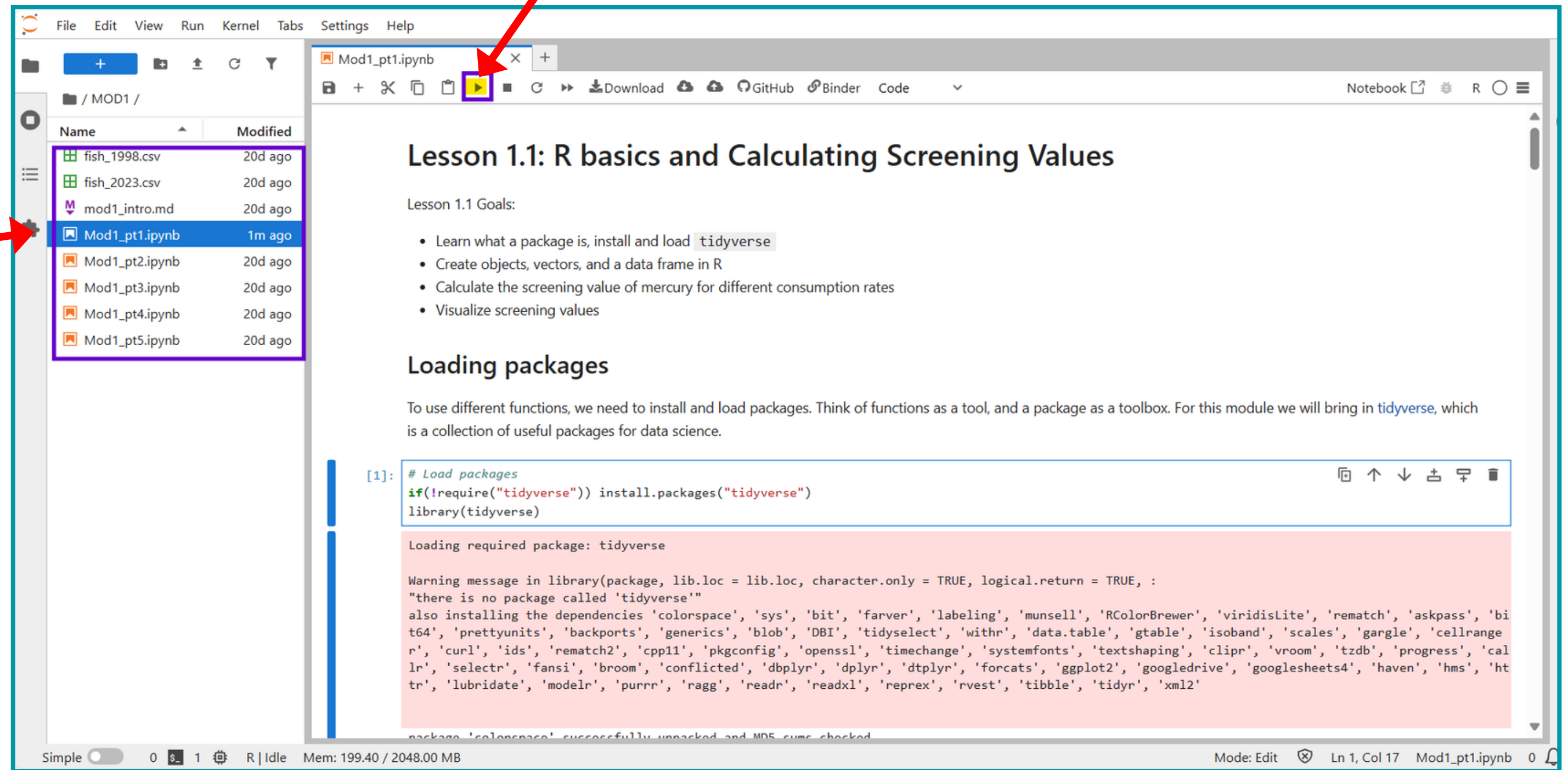
Lesson 1.1 Goals:

- Learn what a package is, install and load `tidyverse`
- Create objects, vectors, and a data frame in R
- Calculate the screening value of mercury for different consumption rates
- Visualize screening values

Familiarize Yourself with the Binder Layout

Run code: press “play” to run the code chunk, or while cursor is in chunk use ctrl+enter to run

pynb: this contains the python notebook code for Module 1. The open page is highlighted in blue. The other files can also be opened to work in.



The screenshot shows the Binder web interface. On the left is a file explorer for the directory '/ MOD1 /'. It lists several files: 'fish_1998.csv', 'fish_2023.csv', 'mod1_intro.md', 'Mod1_pt1.ipynb' (highlighted in blue), 'Mod1_pt2.ipynb', 'Mod1_pt3.ipynb', 'Mod1_pt4.ipynb', and 'Mod1_pt5.ipynb'. A red arrow points from the text 'pynb: this contains the python notebook code for Module 1...' to the 'Mod1_pt1.ipynb' file. On the right is the code editor for 'Mod1_pt1.ipynb'. It shows the title 'Lesson 1.1: R basics and Calculating Screening Values' and a list of goals. Below that is a code chunk with R code to load the 'tidyverse' package. A red arrow points from the text 'Run code: press “play” to run the code chunk...' to the 'play' button in the code editor's toolbar. The code chunk output shows a warning message and a list of dependencies.

```
[1]: # Load packages
if(!require("tidyverse")) install.packages("tidyverse")
library(tidyverse)

Loading required package: tidyverse

Warning message in library(package, lib.loc = lib.loc, character.only = TRUE, logical.return = TRUE, :
"there is no package called 'tidyverse'"
also installing the dependencies 'colorspace', 'sys', 'bit', 'farver', 'labeling', 'munsell', 'RColorBrewer', 'viridisLite', 'rematch', 'askpass', 'bi
t64', 'prettyunits', 'backports', 'generics', 'blob', 'DBI', 'tidyselect', 'withr', 'data.table', 'gtable', 'isoband', 'scales', 'gargle', 'cellrange
r', 'curl', 'ids', 'rematch2', 'cpp11', 'pkgconfig', 'openssl', 'timechange', 'systemfonts', 'textshaping', 'clipr', 'vroom', 'tzdb', 'progress', 'cal
lr', 'selectr', 'fansit', 'broom', 'conflicted', 'dbplyr', 'dplyr', 'dtplyr', 'forcats', 'ggplot2', 'googledrive', 'googlesheets4', 'haven', 'hms', 'ht
tr', 'lubridate', 'modelr', 'purrr', 'ragg', 'readr', 'readxl', 'reprex', 'rvest', 'tibble', 'tidyr', 'xml2'
```

Familiarize Yourself with Code Chunks

Code chunk: an area
with grey
background that
contains code

```
[10]: # Define variables
      weight = 120 # lbs
      days_per_week = 1 # days per week eating fish

      # print the values
      print(weight)
      print(days_per_week)

      [1] 120
      [1] 1
```

Comments:
anything following a
pound sign “#”
within a grey code
chunk; R does NOT
run this code

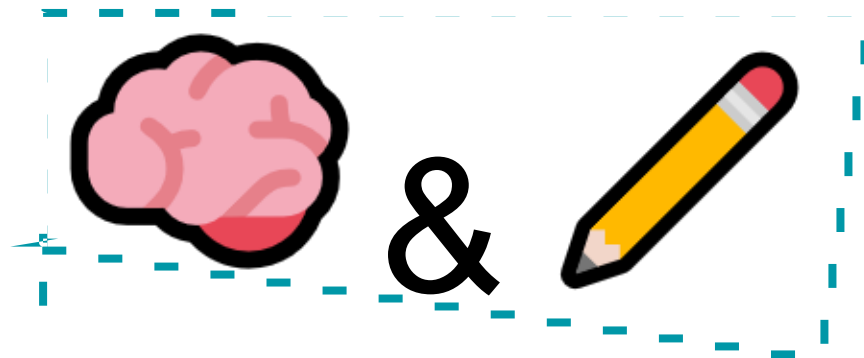
output from
running code

Lesson 1.1: R Basics & Calculating Screening Values



Goals:

- Identify the function name that loads packages
- Recognize objects including variables, vectors, and data frames
- Calculate values using mathematical operations and functions
- Generate a scatterplot using ggplot2
- Describe the relationships between body weight, weekly servings, and screening values (safe limits)



Loading packages

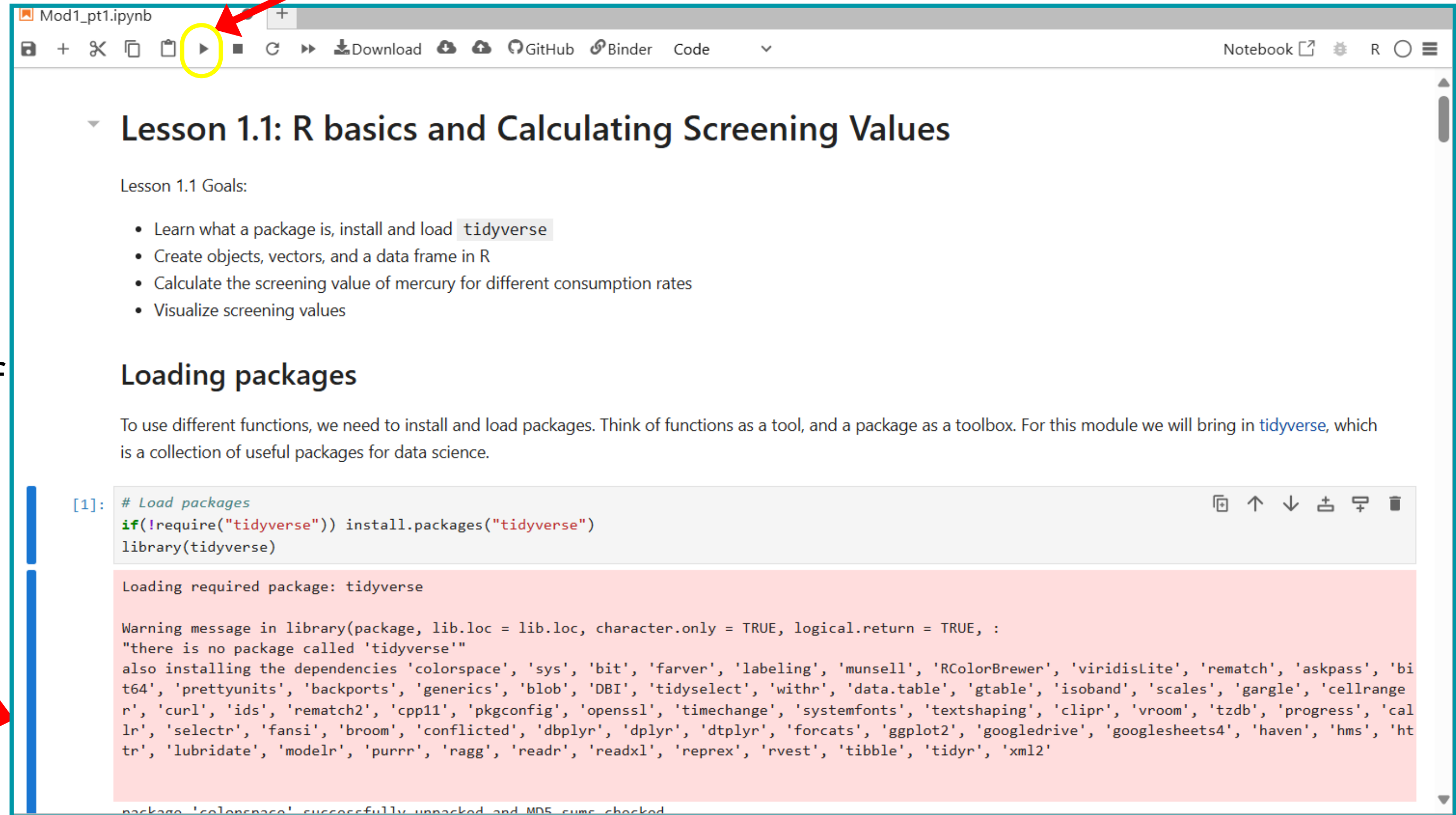
What is the name of the package that we will install and load for this project?

Select Code to Run

3. Run the code for the section of interest.

1. Click in the white space to the left of the code section of interest.

2. The blue vertical line will appear.



Mod1_pt1.ipynb

Lesson 1.1: R basics and Calculating Screening Values

Lesson 1.1 Goals:

- Learn what a package is, install and load `tidyverse`
- Create objects, vectors, and a data frame in R
- Calculate the screening value of mercury for different consumption rates
- Visualize screening values

Loading packages

To use different functions, we need to install and load packages. Think of functions as a tool, and a package as a toolbox. For this module we will bring in `tidyverse`, which is a collection of useful packages for data science.

```
[1]: # Load packages
if(!require("tidyverse")) install.packages("tidyverse")
library(tidyverse)
```

Loading required package: tidyverse

Warning message in library(package, lib.loc = lib.loc, character.only = TRUE, logical.return = TRUE, :
"there is no package called 'tidyverse'"

also installing the dependencies 'colorspace', 'sys', 'bit', 'farver', 'labeling', 'munsell', 'RColorBrewer', 'viridisLite', 'rematch', 'askpass', 'bit64', 'prettyunits', 'backports', 'generics', 'blob', 'DBI', 'tidyselect', 'withr', 'data.table', 'gtable', 'isoband', 'scales', 'gargle', 'cellrange', 'curl', 'ids', 'rematch2', 'cpp11', 'pkgconfig', 'openssl', 'timechange', 'systemfonts', 'textshaping', 'clipr', 'vroom', 'tzdb', 'progress', 'callr', 'selectr', 'fansi', 'broom', 'conflicted', 'dbplyr', 'dplyr', 'dtplyr', 'forcats', 'ggplot2', 'googledrive', 'googlesheets4', 'haven', 'hms', 'httr', 'lubridate', 'modelr', 'purrr', 'ragg', 'readr', 'readxl', 'reprex', 'rvest', 'tibble', 'tidyr', 'xml2'

package 'colorspace' successfully unpacked and MD5 sums checked

Before and After Running a Code Chunk

Mod1_pt1.iipynb

+

Download

GitHub

Binder

Code

Notebook

R

Lesson 1.1: R basics and Calculating Screening Values

Lesson 1.1 Goals:

- Learn what a package is, install and load `tidyverse`
- Create objects, vectors, and a data frame in R
- Calculate the screening value of mercury for different consumption rates
- Visualize screening values

Loading packages

To use different functions, we need to install and load packages. Think of functions as a tool, and a package as a toolbox. For this module we will bring in `tidyverse`, which is a collection of useful packages for data science.

```
[1]: # Load packages
install.packages("tidyverse")
library(tidyverse)
```

Loading required package: tidyverse

Warning message in library(package, lib.loc = lib.loc, character.only = TRUE, logical.return = TRUE, :
"there is no package called 'tidyverse'"

also installing the dependencies 'colorspace', 'sys', 'bit', 'farver', 'labeling', 'munsell', 'RColorBrewer', 'viridisLite', 'rematch', 'askpass', 'bit64', 'prettyunits', 'backports', 'generics', 'blob', 'DBI', 'tidyselect', 'withr', 'data.table', 'gtable', 'isoband', 'scales', 'gargle', 'cellranger', 'curl', 'ids', 'rematch2', 'cpp11', 'pkgconfig', 'openssl', 'timechange', 'systemfonts', 'textshaping', 'clipr', 'vroom', 'tibble', 'progress', 'callr', 'selectr', 'fansi', 'broom', 'conflicted', 'dbplyr', 'dplyr', 'dtplyr', 'forcats', 'ggplot2', 'googledrive', 'googlesheets4', 'haven', 'hms', 'httr', 'lubridate', 'modelr', 'purrr', 'ragg', 'readr', 'readxl', 'reprex', 'rvest', 'tibble', 'tidyr', 'xml2'

package 'colorspace' successfully unpacked and MD5 sums checked

Before Running Code Chunk

```
[1]: # Load packages
if(!require("tidyverse")) install.packages("tidyverse")
library(tidyverse)

Loading required package: tidyverse

— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr 1.1.4 ✓ readr 2.1.4
✓ forcats 1.0.0 ✓ stringr 1.5.1
✓ ggplot2 3.4.4 ✓ tibble 3.2.1
✓ lubridate 1.9.3 ✓ tidyr 1.3.0
✓ purrr 1.0.2
— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag() masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

AFTER Running Code Chunk 1x